





## Syllabus personalizado a sus intereses

- ¿Qué quieren aprender? (que no esté muy apartado del temario)
- Podemos ver – después de aprendizaje no supervisado y procesamiento de textos, y si el tiempo lo permite –:
  - **Árboles de decisión y ensambles de árboles.**  
Clasificación y regresión con datos tabulares; p. ej., predecir si un cliente dejará un servicio.
  - **Métodos de kernel.**  
Clasificación no lineal; p. ej., separar clases que no son linealmente separables.
  - **Descenso de gradiente estocástico.**  
Entrenamiento eficiente con grandes conjuntos de datos; p. ej., modelos con millones de observaciones.
  - **Métodos lineales para regresión/clasificación.**  
Predicción de variables continuas; p. ej., predecir el precio de una vivienda/Predicción de categorías; p. ej., clasificar un correo como spam o no spam. A profundidad, teórico o aplicado rápido.



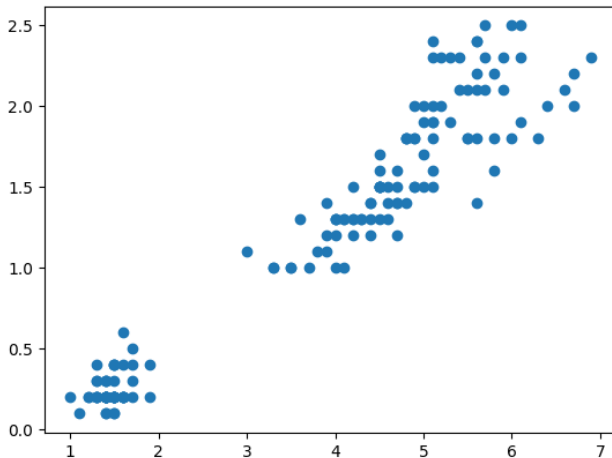


## Tareas del aprendizaje no supervisado

- **Agrupamiento (*clustering*).** Identificar grupos de observaciones similares. **Útil** en: análisis de datos, segmentación de clientes, sistemas de recomendación, motores de búsqueda, segmentación de imágenes, etc.
- **Reducción de dimensiones.** Representar los datos utilizando menos variables, conservando la información más relevante. **Útil** en: maldición de la dimensionalidad, visualización de datos.
- **Detección de anomalías o valores atípicos (*outliers*).** Identificar observaciones que se comportan de manera muy diferente al resto. **Útil** en: detección de fraudes, de productos defectuosos, limpieza de datos (eliminación de *outliers* de un dataset), etc.
- **Estimación de densidad.** Estimar cómo se distribuyen los datos en el espacio de características. **Útil** en: Estimar la función de densidad de probabilidad, detección de anomalías, visualización.

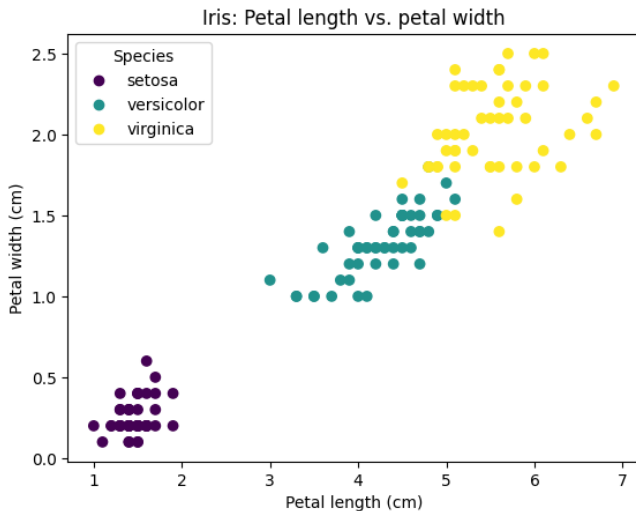
# Clustering

## ¿Cómo agruparían estos datos?



# Clustering

## ¿Cómo agruparían estos datos?

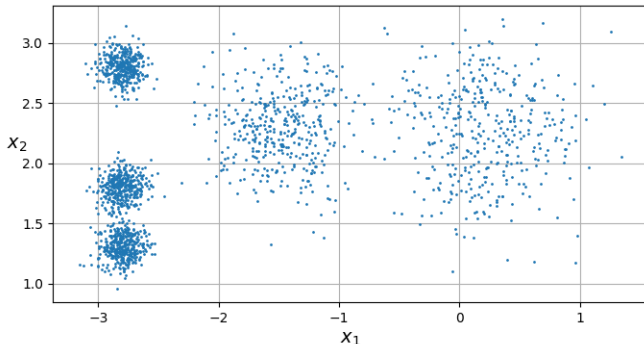


## Aplicaciones

- **Segmentación de clientes:** agrupar clientes según sus compras o comportamiento para personalizar productos, publicidad y recomendaciones.
- **Análisis de datos:** identificar grupos naturales dentro de un conjunto de datos y analizarlos por separado.
- **Reducción de dimensionalidad:** representar cada observación mediante su afinidad o distancia a un conjunto de clusters.
- **Ingeniería de características:** utilizar la pertenencia o afinidad a clusters como nuevas variables para mejorar otros modelos.
- **Detección de anomalías:** identificar observaciones que tienen poca afinidad con todos los clusters.
- **Aprendizaje semisupervisado:** propagar etiquetas conocidas a observaciones similares que pertenecen al mismo cluster.
- **Motores de búsqueda:** agrupar elementos similares, por ejemplo imágenes, para recuperar resultados relacionados.
- **Segmentación de imágenes:** agrupar píxeles similares para simplificar imágenes y facilitar la detección de objetos.

## $K$ -means (1957)

- Supongamos que queremos agrupar en  $K$  clusters.
- $K$ -means intentará obtener una distribución óptima de cómo ubicar  $K$  centros en los datos.
- Queremos que la distancia intra-clusters sea pequeña y la distancia inter-clusters sea grande.





## Función objetivo

K-means minimiza la suma de distancias cuadradas dentro de cada cluster (*medida de distorsión*):

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2.$$

donde

$$r_{nk} = \begin{cases} 1, & \text{si } \mathbf{x}_n \text{ pertenece al cluster } k, \\ 0, & \text{en otro caso.} \end{cases}$$

**Meta:** Encontrar  $\{r_{nk}\}$  y  $\{\boldsymbol{\mu}_k\}$  que minimicen  $J$ .

## ¿Cómo minimizar $J$ ?

Haciendo un proceso iterativo de dos pasos:

1. Teniendo los  $\mu_k$ s, minimizamos  $J$  con respecto a los  $r_{nk}$ s.
2. Teniendo los  $r_{nk}$ s, minimizamos  $J$  con respecto a los  $\mu_k$ s.

Hasta que el algoritmo converja, es decir hasta que no cambien los valores.

## Paso 1: asignación de clusters (encontrar $\{r_{nk}\}$ )

Si los centroides  $\mu_1, \dots, \mu_K$  están fijos, para optimizar  $J$  con respecto a los  $r_{nk}$ s,

## Paso 1: asignación de clusters (encontrar $\{r_{nk}\}$ )

Si los centroides  $\mu_1, \dots, \mu_K$  están fijos, para optimizar  $J$  con respecto a los  $r_{nk}$ s, asignamos cada punto al centroide más cercano:

$$r_{nk} = \begin{cases} 1, & \text{si } k = \arg \min_j \|\mathbf{x}_n - \mu_j\|^2, \\ 0, & \text{en otro caso.} \end{cases}$$

Es decir, cada observación se asigna al cluster cuyo centroide está más cerca.

## Paso 2: actualización de centroides (encontrar $\{\mu_k\}$ )

Si las asignaciones  $r_{nk}$  están fijas, para optimizar  $J$  con respecto a los  $\mu_k$ s,

## Paso 2: actualización de centroides (encontrar $\{\mu_k\}$ )

Si las asignaciones  $r_{nk}$  están fijas, para optimizar  $J$  con respecto a los  $\mu_k$ s, actualizamos cada centroide como el promedio de los puntos asignados a ese cluster:

$$\mu_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}.$$

Por eso el centroide representa el “centro” de los puntos del cluster.

## Algoritmo K-means

1. Elegir el número de clusters  $K$ .
2. Inicializar los centroides  $\mu_1, \dots, \mu_K$ .
3. Asignar cada punto al centroide más cercano.
4. Recalcular cada centroide como el promedio de sus puntos.
5. Repetir los pasos 3 y 4 hasta que las asignaciones ya no cambien.

# Convergencia

- En cada paso, la función objetivo  $J$  no aumenta.
- El paso de asignación reduce  $J$  al escoger el centroide más cercano.
- El paso de actualización reduce  $J$  al mover el centroide al promedio.
- Por lo tanto, el algoritmo converge.

**Importante:** K-means puede converger a un mínimo local, por lo que la inicialización importa.

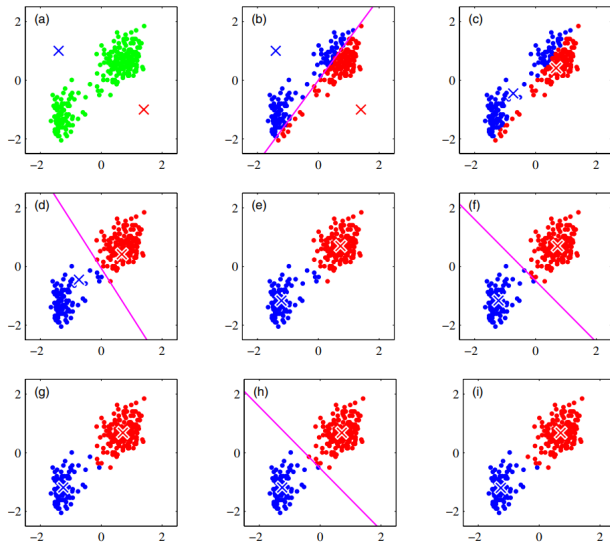
## Interpretación geométrica

- K-means divide el espacio en regiones.
- Cada región contiene los puntos más cercanos a un mismo centroide.
- Los centroides se mueven hasta quedar en el centro de sus respectivos clusters.

El algoritmo alterna entre:

asignar puntos  $\longleftrightarrow$  mover centroides.

## Interpretación geométrica



## Distancia Euclidiana

¿Y si no quiero/puedo usar la distancia euclidiana?

Supongamos que tenemos una medida de disimilitud  $\mathcal{V}(\mathbf{x}, \mathbf{x}')$ . Podemos minimizar la siguiente medida de distorsión:

$$\tilde{J} = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \mathcal{V}(\mathbf{x}_n, \boldsymbol{\mu}_k),$$

que da lugar al algoritmo de  $K$ -medoides: teniendo los centros  $\boldsymbol{\mu}_k$ , para obtener los  $r_{nk}$ s, se asigna cada punto al cluster para el cual su disimilitud es la menor. Para el caso contrario, se restringe a que los centros sólo puedan ser puntos de los datos.

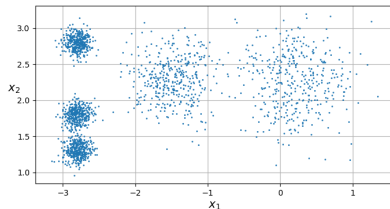
# K-means en Scikit-Learn

Supongamos que tengo estos datos:

```
import numpy as np
from sklearn.datasets import make_blobs

blob_centers = np.array([[ 0.2, 2.3], [-1.5 , 2.3], [-2.8, 1.8], [-2.8,
    2.8], [-2.8, 1.3]])
blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1])
X, y = make_blobs(n_samples=2000, centers=blob_centers, cluster_std=
    blob_std, random_state=7)

plt.scatter(X[:, 0], X[:, 1], s=1)
```



# K-means en Scikit-Learn

Usando K-means:

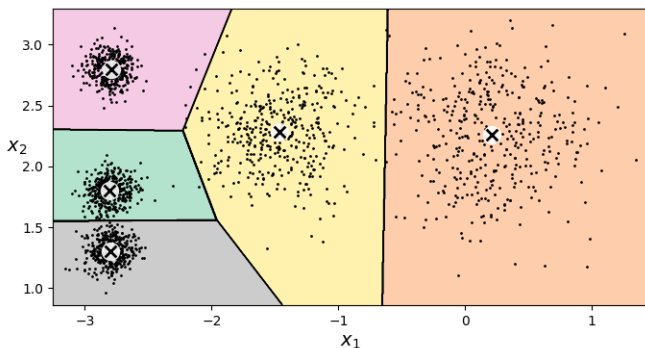
```
from sklearn.cluster import KMeans

k = 5
kmeans = KMeans(n_clusters=k, random_state=43)
y_pred = kmeans.fit_predict(X)

# or just kmeans.fit(X)

kmeans.cluster_centers_
```

## K-means en Scikit-Learn



Si tengo nuevos puntos:

```
X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])  
kmeans.predict(X_new)
```

## Sensible a inicialización

Probar con semilla 42.

## Sensible a inicialización

Probar con semilla 42.

Existen varios métodos para inicializar los centroides.

## Inercia

Podemos correr  $K$ -means con diferentes semillas y quedarnos con el mejor cluster...

¿Cuál es el mejor cluster?

# Inercia

Podemos correr  $K$ -means con diferentes semillas y quedarnos con el mejor cluster...

¿Cuál es el mejor cluster?

¡Necesitamos definir una métrica! La *inercia* está definida como:

$$\text{inercia} = \sum_k^K \sum_n^N r_{nk} \|x_n - \mu_j\|^2,$$

y es la suma de todas las distancias al cuadrado entre cada punto y su centroide más cercano.

Fórmula más intuitiva:

$$\text{inercia} = \sum_k^K \sum_{x \in C_k} \|x - \mu_k\|^2,$$

## Inercia en Scikit-Learn

```
kmeans.inertia_
```

## Inicialización con *k-means++*

La inicialización *k-means++* busca elegir centroides iniciales bien separados entre sí.

1. Elegir el primer centroide  $\mu_1$  de forma aleatoria del conjunto de datos.
2. Para elegir un nuevo centroide  $\mu_i$ , seleccionar un punto  $\mathbf{x}_i$  con probabilidad

$$\frac{D(\mathbf{x}_i)^2}{\sum_{n=1}^N D(\mathbf{x}_n)^2},$$

donde  $D(\mathbf{x}_i)$  es la distancia entre  $\mathbf{x}_i$  y el centroide ya seleccionado más cercano.

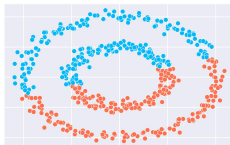
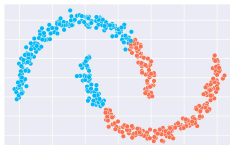
3. Repetir el paso anterior hasta haber seleccionado los  $K$  centroides iniciales  $\mu_1, \dots, \mu_K$ .

**Idea:** los puntos más alejados de los centroides ya elegidos tienen mayor probabilidad de convertirse en nuevos centroides.

## Ventajas y desventajas de $K$ -means

- + Es rápido
- + Escala fácil
- Seleccionar  $K$  no es una tarea fácil.
- Tiene simetría esférica. Clusters de diferentes formas pueden dar malos resultados:

**KMeans**



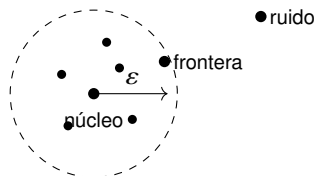
## DBSCAN

**DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*) es un método de agrupamiento basado en densidad. Parámetros:  $\epsilon$ , `min_samples`.

- Para cada observación, se cuentan los puntos que se encuentran dentro de una distancia  $\epsilon$ .
- Esta región se conoce como la **vecindad- $\epsilon$**  de la observación.
- Si una observación tiene al menos `min_samples` puntos en su vecindad- $\epsilon$  (incluyéndose a sí misma), se considera un **punto núcleo** (*core point*).
- Los puntos que pertenecen a la vecindad de un punto núcleo forman parte del mismo **cluster**. Una cadena de puntos núcleo vecinos puede formar un solo cluster.
- Un punto que no es núcleo y que tampoco se encuentra en la vecindad de ningún punto núcleo se considera **ruido o anomalía**.

## DBSCAN: tipos de puntos

- **Punto núcleo (*core point*):**  
Tiene al menos `min_samples` observaciones dentro de su vecindad- $\epsilon$ .
- **Punto frontera (*border point*):**  
No tiene suficientes vecinos para ser núcleo, pero se encuentra dentro de la vecindad- $\epsilon$  de un punto núcleo.
- **Ruido (*noise*):**  
No es punto núcleo ni se encuentra en la vecindad de uno.



# ¡Gracias!